

\$match (aggregation stage)

Definition

`$match` Filters documents based on a specified query predicate. Matched documents are passed to the next pipeline stage.

Compatibility

`$match` You can use `$match` for deployments hosted in the following environments:

- MongoDB Atlas: The fully managed service for MongoDB deployments in the cloud
- MongoDB Enterprise: The subscription-based, self-managed version of MongoDB
- MongoDB Community: The source-available, free-to-use, and self-managed version of MongoDB

Syntax

```
{ $match: { <query predicate> } }
```

The syntax for the `$match` query predicate is identical to the syntax used in the query argument of a `find()` command.

Behavior

Pipeline Optimization

- Place the `$match` as early in the aggregation pipeline as possible. Because `$match` limits the total number of documents in the aggregation pipeline, earlier `$match` operations minimize the amount of processing down the pipe.
- If you place a `$match` at the very beginning of a pipeline, the query can take advantage of indexes like any other `db.collection.find()` or `db.collection.findOne()`.

Expressions in Query Predicates

To include expressions in a query predicate, use the `$expr` operator.

0, Null, False or Missing Values

A `$match` stage filters out a document from pipeline results if one of the following conditions applies:

- The `$match` query predicate returns a `0`, `null`, or `false` value on that document.
- The `$match` query predicate uses a field that is missing from that document.

Restrictions

- You cannot use `$where` in a `$match` stage.
- You cannot use `$near` or `$nearSphere` in a `$match` stage. As an alternative, you can either:
 - Use the `$geoNear` stage instead of the `$match` stage.
 - Use the `$geoWithin` query predicate operator with `$center` or `$centerSphere` in the `$match` stage.
- To use `$text` in a `$match` stage, the `$match` stage has to be the first stage of the pipeline.

Views do not support `$text`.

`$text` provides text query capabilities for self-managed (non-Atlas) deployments. For data hosted on MongoDB, MongoDB also offers an improved full-text query solution, MongoDB Search.

Filter Data on Atlas by Using MongoDB Search

For data stored in MongoDB Atlas, you can use the MongoDB Search compound Operator operator `filter` option to match or filter documents when running `$search` queries. Running `$match` after `$search` is less performant than running `$search` with the compound Operator operator `filter` option.

To learn more about the `filter` option, see compound Operator in the Atlas documentation.

Examples

The examples use a collection named `articles` with the following documents:

```
{ "_id" : ObjectId("512bc95fe835e68f199c8686"), "author" : "dave", "score" : 80,
{ "_id" : ObjectId("512bc962e835e68f199c8687"), "author" : "dave", "score" : 85,
{ "_id" : ObjectId("55f5a192d4bede9ac365b257"), "author" : "ahn", "score" : 60,
{ "_id" : ObjectId("55f5a192d4bede9ac365b258"), "author" : "li", "score" : 55,
{ "_id" : ObjectId("55f5a1d3d4bede9ac365b259"), "author" : "annT", "score" : 60,
{ "_id" : ObjectId("55f5a1d3d4bede9ac365b25a"), "author" : "li", "score" : 94,
{ "_id" : ObjectId("55f5a1d3d4bede9ac365b25b"), "author" : "ty", "score" : 95,
```

Equality Match

The following operation uses `$match` to perform an equality match:

```
db.articles.aggregate(
  [ { $match : { author : "dave" } } ]
);
```

The `$match` selects the documents where the `author` field equals `dave`, and the aggregation returns the following:

```
{ "_id" : ObjectId("512bc95fe835e68f199c8686"), "author" : "dave", "score" : 80,
{ "_id" : ObjectId("512bc962e835e68f199c8687"), "author" : "dave", "score" : 85,
```

Perform a Count

The following example selects documents to process using the `$match` pipeline operator and then pipes the results to the `$group` pipeline operator to compute a count of the documents:

```
db.articles.aggregate( [
  { $match: { $or: [ { score: { $gt: 70, $lt: 90 } }, { views: { $gte: 1000 } } }
  { $group: { _id: null, count: { $sum: 1 } } }
] );
```

In the aggregation pipeline, `$match` selects the documents where either the `score` is greater than `70` and less than `90` or the `views` is greater than or equal to `1000`. These documents are then piped to the `$group` to perform a count. The aggregation returns the following:

```
{ "_id" : null, "count" : 5 }
```

Match Array Elements

To filter documents from a collection or previous aggregation stage based on elements in an array, use the `$elemMatch` operator in the query predicate of the `$match` stage:

```
db.aggregate( [
  {
    $documents: [
      { student_id: 1, scores: [ 0.75, 0.65, 0.73 ] },
      { student_id: 2, scores: [ 0.9, 0.88, 0.98 ] },
      { student_id: 3, scores: [ 0.9, 0.84, 0.93 ] }
    ]
  }, {
    $match: {
      scores: { $elemMatch: { $gte: 0.9 } }
    }
  }
] )
```

```
[
  { student_id: 2, scores: [ 0.9, 0.88, 0.98 ] },
  { student_id: 3, scores: [ 0.9, 0.84, 0.93 ] }
]
```

The C# examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB .NET/C# Driver documentation](#).

The following `Movie` class models the documents in the `sample_mflix.movies` collection:

```

public class Movie
{
    public ObjectId Id { get; set; }

    public int Runtime { get; set; }

    public string Title { get; set; }

    public string Rated { get; set; }

    public List<string> Genres { get; set; }

    public string Plot { get; set; }

    public ImdbData Imdb { get; set; }

    public int Year { get; set; }

    public int Index { get; set; }

    public string[] Comments { get; set; }

    [BsonElement("lastupdated")]
    public DateTime LastUpdated { get; set; }
}

```

The C# classes on this page use Pascal case for their property names, but the field names in the MongoDB collection use camel case. To account for this difference, you can use the following code to register a `ConventionPack` when your application starts:

```

var camelCaseConvention = new ConventionPack { new CamelCaseElementNameConvention() };
ConventionRegistry.Register("CamelCase", camelCaseConvention, type => true);

```

```
$match
```

```
Match()
```

matches all `Movie` documents where the `Title` field is equal to `"The Shawshank Redemption"`:

To use the MongoDB .NET/C# driver to add a `$match` stage to an aggregation pipeline, call the `Match()` method on a `PipelineDefinition` object.

The following example creates a pipeline stage that matches all `Movie` documents where the `Title` field is equal to `"The Shawshank Redemption"`:

```
var pipeline = new EmptyPipelineDefinition<Movie>()  
    .Match(m => m.Title == "The Shawshank Redemption");
```

The Node.js examples on this page use the `sample_mflix` database from the Atlas sample datasets. To learn how to create a free MongoDB Atlas cluster and load the sample datasets, see [Get Started in the MongoDB Node.js driver documentation](#).

`$match`

matches all `movie` documents where the `title` field is equal to `"The Shawshank Redemption"`

To use the MongoDB Node.js driver to add a `$match` stage to an aggregation pipeline, use the `$match` operator in a pipeline object.

The following example creates a pipeline stage that matches all `movie` documents where the `title` field is equal to `"The Shawshank Redemption"`. The example then runs the aggregation pipeline:

```
const pipeline = [  
  {  
    $match: {  
      title: "The Shawshank Redemption"  
    }  
  }  
];  
  
const cursor = collection.aggregate(pipeline);  
return cursor;
```

Learn More

Refer to the [Complete Aggregation Pipeline Tutorials](#) for more information and use cases on aggregation.